

프로세스에 입각한 바이트코딩의 효과 분석 보고서

전자전기융합공학과 C093020 김규표

1. 서론

최근 소프트웨어 개발 과정에서 생성형 인공지능을 활용하여 코드를 작성하고 수정하는 방식이 점차 확산되고 있다. 이러한 개발 방식은 일반적으로 바이트코딩이라고 불리며, 개발자가 자연어로 요구사항이나 문제 상황을 설명하면 인공지능이 코드, 오류 수정 방향, 설계 아이디어 등을 제안하는 방식으로 진행된다. 바이트코딩은 초기 구현 속도를 높이고 개발자가 익숙하지 않은 기술을 빠르게 활용할 수 있게 한다는 장점이 있다. 그러나 인공지능이 생성한 결과물을 그대로 사용하는 경우 요구사항과 실제 구현이 어긋나거나, 테스트와 품질 관리가 부족해질 수 있다는 한계도 존재한다.

본 보고서는 바이트코딩을 단순한 코드 생성 도구로 사용하는 것이 아니라, 요구사항 분석, 설계, 구현, 테스트, 품질 관리에 이르는 소프트웨어 개발 프로세스 안에서 어떻게 활용할 수 있는지를 분석하는 것을 목적으로 한다. 이를 위해 Python과 NiceGUI를 활용하여 대학생 과제 관리 웹앱을 개발하였으며, 개발 과정에서 ChatGPT를 바이트코딩 도구로 사용하였다. 개발 과정은 GitHub를 통해 관리하였고, discussion-log와 prompt-log를 작성하여 과제 착수 시점부터 지속적으로 논의와 개선이 이루어졌음을 증빙하였다.

2. 프로젝트 개요

본 프로젝트의 개발 대상은 대학생을 위한 과제 관리 웹앱이다. 대학생은 여러 과목의 과제를 동시에 관리해야 하며, 과제 제목, 설명, 마감일, 완료 여부를 확인할 수 있는 간단한 도구가 필요하다. 이에 따라 본 프로젝트에서는 사용자가 과제를 등록하고, 등록된 과제를 목록에서 확인하며, 완료 여부를 변경하고, 필요 없는 과제를 삭제할 수 있는 웹앱을 구현하였다.

개발 기술로는 Python과 NiceGUI를 사용하였다. 별도의 데이터베이스는 사용하지 않고, 과제 정보는 tasks.json 파일에 저장하도록 구성하였다. 이를 통해 개발 환경을 단순화하고, 웹앱의 핵심 기능인 과제 등록, 조회, 완료 처리, 삭제, 필터링 기능에 집중하였다.

본 프로젝트의 주요 기능은 다음과 같다.

- 과제 추가
- 과제 목록 조회
- 과제 마감일 관리
- 과제 완료 처리
- 과제 삭제
- 전체/미완료/완료 필터링
- 입력값 검증
- 포트 충돌 발생 시 다른 포트로 실행

본 프로젝트에서 사용한 바이브코딩 도구는 ChatGPT이다. ChatGPT는 초기 코드 작성, 오류 원인 분석, UI 개선 방향 제안, README 작성, 테스트 결과 정리, discussion-log 및 prompt-log 작성에 활용되었다. 다만 최종 기능 판단, 실행 결과 확인, 스크린샷 정리, 제출 자료 구성은 개발자가 직접 수행하였다.

3. 개발 프로세스 적용

3.1 요구사항 분석

개발 초기 단계에서는 대학생 과제 관리 웹앱에 필요한 기능을 중심으로 요구사항을 정리하였다. 사용자가 과제를 입력하기 위해서는 과제 제목, 설명, 마감일을 입력할 수 있어야 하며, 등록된 과제는 목록 형태로 표시되어야 한다. 또한 과제가 완료되었는지 여부를 관리할 수 있어야 하고, 완료된 과제와 미완료 과제를 구분해서 확인할 수 있어야 한다.

요구사항 분석 단계에서 정리한 주요 기능 요구사항은 다음과 같다.

첫째, 사용자는 과제 제목, 설명, 마감일을 입력하여 과제를 추가할 수 있어야 한다. 둘째, 등록된 과제는 목록으로 표시되어야 하며, 제목, 설명, 마감일, 완료 상태가 함께 표시되어야 한다. 셋째, 사용자는 과제를 완료 처리하거나 다시 미완료 상태로 변경할 수 있어야 한다. 넷째, 필요 없는 과제는 삭제할 수 있어야 한다. 다섯째, 전체, 미완료, 완료 필터를 통해 과제 목록을 구분해서 확인할 수 있어야 한다. 여섯째, 필수 입력값이 비어 있는 경우 과제가 등록되지 않도록 입력값 검증이 이루어져야 한다.

이 단계에서 바이트코딩은 요구사항을 구체화하는 데 도움이 되었다. 처음에는 단순히 과제 관리 웹앱을 만든다는 수준의 아이디어였지만, ChatGPT와의 질의응답을 통해 필요한 기능을 목록화하고, 어떤 화면을 캡처해야 하는지까지 정리할 수 있었다. 그러나 요구사항의 우선순위와 최종 기능 범위는 개발자가 직접 판단해야 했다. 인공지능은 가능한 기능을 많이 제안할 수 있지만, 과제 규모와 제출 조건에 맞는 적절한 범위를 결정하는 것은 개발자의 역할이었다.

3.2 설계

설계 단계에서는 화면 구조와 데이터 구조를 정리하였다. 화면은 크게 과제 입력 영역과 과제 목록 영역으로 구성하였다. 과제 입력 영역에서는 제목, 설명, 마감일을 입력하고 과제 추가 버튼을 누를 수 있도록 하였다. 과제 목록 영역에서는 등록된 과제를 카드 또는 목록 형태로 표시하고, 각 과제마다 완료 처리 버튼과 삭제 버튼을 제공하도록 하였다.

데이터 구조는 JSON 파일 저장 방식을 사용하였다. 각 과제는 제목, 설명, 마감일, 완료 상태를 포함하는 객체 형태로 저장된다. 데이터베이스를 사용하지 않은 이유는 본 프로젝트의 목적이 대규모 서비스 구현이 아니라, 바이트코딩을 활용한 개발 프로세스 분석이기 때문이다. 따라서 파일 기반 저장 방식을 통해 구현 복잡도를 낮추고, 요구사항 분석부터 품질 관리까지의 과정에 집중하였다.

설계 단계에서 중요한 점은 필터 상태 관리였다. 전체, 미완료, 완료 버튼을 누르면 현재 선택된 필터에 따라 표시되는 과제 목록이 달라져야 한다. 또한 선택된 필터가 화면에서 명확하게 구분되어야 사용자가 현재 어떤 목록을 보고 있는지 쉽게 이해할 수 있다.

이를 위해 현재 선택된 필터 상태를 변수로 관리하고, 필터 버튼 영역을 다시 렌더링하는 방식으로 UI를 구성하였다.

바이브코딩은 화면 구성과 데이터 구조를 빠르게 설계하는 데 도움이 되었다. 하지만 UI의 사용성 문제는 실제 실행 화면을 확인한 뒤에야 명확히 드러났다. 예를 들어 필터 버튼이 모두 비슷한 색상으로 표시되면 기능은 정상적으로 동작하더라도 사용자가 현재 선택 상태를 직관적으로 파악하기 어렵다. 따라서 설계 단계에서는 기능 동작뿐만 아니라 사용자 관점의 가독성도 함께 고려해야 한다는 점을 확인하였다.

3.3 구현

구현 단계에서는 Python과 NiceGUI를 사용하여 웹앱을 작성하였다. NiceGUI는 Python 코드만으로 웹 기반 UI를 구성할 수 있어, 비교적 짧은 시간 안에 실행 가능한 웹앱을 구현하는 데 적합하였다. ChatGPT를 활용하여 초기 코드 구조를 생성하고, 실행 중 발생한 오류나 기능 추가 요청을 반영하면서 코드를 수정하였다.

구현 과정에서는 과제 추가, 목록 출력, 완료 처리, 삭제, 필터링 기능을 순차적으로 구현하였다. 과제를 추가하면 입력된 정보가 tasks.json 파일에 저장되고, 화면에 과제 목록이 갱신되도록 하였다. 완료 버튼을 누르면 과제의 완료 상태가 변경되고, 삭제 버튼을 누르면 해당 과제가 목록에서 제거되도록 하였다. 필터 버튼은 전체, 미완료, 완료로 구성하였으며, 선택된 필터에 따라 표시되는 과제 목록이 달라지도록 구현하였다.

구현 중 발생한 문제 중 하나는 포트 충돌이었다. NiceGUI 기본 포트가 이미 사용 중인 경우 앱이 정상적으로 실행되지 않았기 때문에, `ui.run(port=8090)`과 같이 다른 포트를 지정하여 실행하는 방법을 적용하였다. 이 내용은 README의 실행 방법에도 포함하여 같은 문제가 발생했을 때 해결할 수 있도록 정리하였다.

또한 필터 버튼 UI도 개선하였다. 초기에는 전체, 미완료, 완료 필터 버튼의 선택 상태가 명확하게 구분되지 않았다. 이후 버튼 색상을 파란색 계열로 변경하고, 현재 선택된 버튼만 더 진한 색상 또는 강조된 테두리로 표시되도록 수정하였다. 이를 통해 사용자가 현재 선택된 필터를 쉽게 인식할 수 있도록 하였다.

이 단계에서 바이브코딩은 코드 생성과 오류 수정 속도를 높이는 데 효과적이었다. 하지만 생성된 코드를 그대로 사용하는 것만으로는 충분하지 않았다. 실제 실행 결과를 확인하고, 의도한 기능이 제대로 동작하는지 검증하며, UI가 사용하기 쉬운지 판단하는 과정이 필요했다. 즉, 바이브코딩은 구현을 자동으로 완성해 주는 방식이라기보다 개발자의 판단을 보조하는 방식으로 활용될 때 효과적이었다.

3.4 테스트

테스트 단계에서는 구현된 기능이 요구사항에 맞게 동작하는지 확인하였다. 테스트 항목은 빈 목록 표시, 과제 추가, 날짜 선택, 완료 처리, 전체 필터, 미완료 필터, 완료 필터, 입력값 검증, 삭제 기능, 포트 변경 실행으로 구성하였다. 각 테스트는 실제 웹앱 실행 화면을 통해 확인하였고, 결과를 표 형식으로 정리하였다.

기능별 테스트 결과는 다음과 같다.

테스트 항목	테스트 내용	예상 결과	실제 결과
--------	--------	-------	-------

---	---	---	---
-----	-----	-----	-----

빈 목록 표시	등록된 과제가 없는 상태에서 과제 목록을 확인한다.	표시할 과제가 없다는 안내 문구가 출력된다.	정상 동작
---------	------------------------------	--------------------------	-------

과제 추가	제목, 설명, 마감일을 입력한 뒤 과제 추가 버튼을 클릭한다.	입력한 과제가 목록에 추가된다.	정상 동작
-------	------------------------------------	-------------------	-------

날짜 선택	달력 UI에서 마감일을 선택한다.	선택한 날짜가 마감일로 반영된다.	정상 동작
-------	--------------------	--------------------	-------

완료 처리	미완료 상태의 과제에서 완료 버튼을 클릭한다.	과제 상태가 완료로 변경된다.	정상 동작
-------	---------------------------	------------------	-------

전체 필터	전체 버튼을 클릭한다.	완료 및 미완료 과제가 모두 표시된다.	정상 동작
-------	--------------	-----------------------	-------

미완료 필터	미완료 버튼을 클릭한다.	미완료 상태의 과제만 표시된다.	정상 동작
--------	---------------	-------------------	-------

완료 필터	완료 버튼을 클릭한다.	완료 상태의 과제만 표시된다.	정상 동작
-------	--------------	------------------	-------

입력값 검증	필수 입력값을 비운 상태에서 과제 추가를 시도한다.	과제가 추가되지 않고 검증 메시지가 표시된다.	정상 동작
--------	------------------------------	---------------------------	-------

삭제 기능	과제의 삭제 버튼을 클릭한다.	해당 과제가 목록에서 제거된다.	정상 동작
-------	------------------	-------------------	-------

포트 변경 실행	기본 포트 충돌 시 8090 포트로 실행한다.	지정한 포트에서 앱이 실행된다.	정상 동작
----------	---------------------------	-------------------	-------

테스트 과정에서 단순히 기능이 실행되는지만 보는 것이 아니라, 제출 자료로 활용할 수 있도록 각 기능의 실행 화면을 캡처하였다. 캡처 항목은 빈 목록 화면, 과제 추가 화면, 날짜 선택 화면, 완료 처리 화면, 전체 필터 화면, 미완료 필터 화면, 완료 필터 화면, 입력값 검증 화면, 삭제 기능 화면으로 구성하였다.

테스트 단계에서 얻은 교훈은 바이브코딩으로 생성한 코드도 반드시 체계적인 검증이 필요하다는 점이다. 인공지능이 제안한 코드는 문법적으로 맞더라도, 요구사항을 완전히 만족하는지 또는 사용자가 이해하기 쉬운 화면인지까지 보장하지 않는다. 따라서 요구사항별 테스트 항목을 만들고, 예상 결과와 실제 결과를 비교하는 과정이 필요하다.

3.5 품질 관리

품질 관리 단계에서는 기능의 정상 동작뿐만 아니라 사용성, 가독성, 제출 자료의 완성도를 함께 점검하였다. 가장 대표적인 개선 사항은 필터 버튼의 선택 상태 표시였다. 초기 화면에서는 전체, 미완료, 완료 버튼의 색상 차이가 크지 않아 현재 어떤 필터가 선택되었는지 구분하기 어려웠다. 이후 파란색 계열의 버튼 스타일을 적용하고, 선택된 버튼만 더 강하게 강조하여 사용자가 현재 상태를 쉽게 파악할 수 있도록 개선하였다.

입력값 검증도 품질 관리의 중요한 요소였다. 필수 입력값이 비어 있는 상태에서 과제가 등록되면 데이터의 신뢰성이 떨어지기 때문에, 빈 입력값을 방지하는 검증 기능을 테스트 항목에 포함하였다. 또한 포트 충돌 문제에 대한 해결 방법을 README에 작성하여 실행 환경이 달라져도 사용자가 앱을 실행할 수 있도록 하였다.

형상 관리는 GitHub를 통해 수행하였다. 코드, README, discussion-log, prompt-log, screenshots 폴더를 저장소에 정리하여 업로드하였다. 이를 통해 단순히 최종 결과물만 제출하는 것이 아니라, 개발 과정에서 어떤 논의와 수정이 이루어졌는지 증빙할 수 있도록 하였다.

품질 관리 단계에서 바이브코딩은 문서 작성과 개선 방향 정리에 도움이 되었다. 하지만 품질 기준을 세우고, 어떤 화면을 제출 자료로 사용할지 결정하며, 기능별 테스트 결과를 최종적으로 판단하는 역할은 개발자에게 있었다. 따라서 바이브코딩 환경에서도 개발 프로세스와 품질 관리 활동은 여전히 중요하다.

4. 바이브코딩의 효과 분석

본 프로젝트를 통해 확인한 바이브코딩의 가장 큰 효과는 개발 속도 향상이다. 개발자가 자연어로 필요한 기능을 설명하면 인공지능이 초기 코드 구조를 제안하고, 오류 발생 시 수정 방향을 제시하기 때문에 처음부터 모든 코드를 직접 작성하는 것보다 빠르게 실행 가능한 결과물을 만들 수 있었다. 특히 NiceGUI처럼 익숙하지 않은 도구를 사용할 때도 기본 구조를 빠르게 파악하고 기능을 구현하는 데 도움이 되었다.

두 번째 효과는 문제 해결 과정의 단축이다. 포트 충돌이나 UI 표시 문제처럼 개발 과정에서 발생하는 문제를 설명하면, 인공지능이 가능한 원인과 해결 방법을 제시하였다. 이를 통해 오류 원인을 찾는 시간을 줄일 수 있었다. 예를 들어 기본 포트가 이미 사용 중인 경우 다른 포트를 지정하여 실행하는 방법을 적용하였고, 필터 버튼의 선택 상태가 명확하지 않은 문제는 색상과 테두리 스타일을 수정하는 방식으로 개선하였다.

세 번째 효과는 문서화 지원이다. 본 프로젝트에서는 README, discussion-log, prompt-log, 기능별 테스트 결과 표, 보고서 초안 작성에 바이브코딩을 활용하였다. 개발 과정에서 수행한 내용을 문서로 정리하는 작업은 시간이 많이 소요되지만, 인공지능을 활용하면 구조화된 초안을 빠르게 작성할 수 있다. 이후 개발자가 실제 작업 내용에 맞게 수정하면 제출 자료의 완성도를 높일 수 있다.

그러나 바이브코딩에는 한계도 있었다. 인공지능은 사용자의 요구를 바탕으로 코드를 생성하지만, 과제의 평가 기준이나 실제 제출 조건을 완전히 이해하고 판단하는 것은 아니다. 따라서 개발자가 요구사항을 명확히 제시하지 않으면 불필요한 기능이 추가되거나, 제출 조건과 맞지 않는 방향으로 결과물이 만들어질 수 있다. 또한 생성된 코드가 정상적으로 보이더라도 실제 실행 환경에서 오류가 발생할 수 있으므로 반드시 테스트가 필요하다.

결국 바이브코딩은 개발 프로세스를 대체하는 것이 아니라, 각 단계에서 개발자의 작업을 보조하는 도구로 활용될 때 효과적이다. 요구사항 분석 단계에서는 기능 범위를 구체화하는 데 도움을 주고, 설계 단계에서는 화면 구조와 데이터 구조를 제안하며, 구현 단계에서는 코드 초안을 생성한다. 테스트 단계에서는 테스트 항목을 정리하고, 품질 관리 단계에서는 개선 방향을 제시한다. 그러나 최종 판단과 검증은 개발자가 수행해야 한다.

5. 프로세스 적용의 교훈

본 프로젝트를 통해 얻은 첫 번째 교훈은 요구사항 분석의 중요성이다. 바이트코딩을 사용할 때 요구사항이 모호하면 인공지능이 생성한 결과도 모호해진다. 예를 들어 단순히 "과제 관리 앱을 만들어줘"라고 요청하는 것보다, 과제 추가, 완료 처리, 삭제, 필터링, 입력값 검증 등 구체적인 기능을 명시할 때 더 적절한 결과를 얻을 수 있었다. 따라서 바이트코딩에서도 요구사항을 명확히 정의하는 과정이 필수적이다.

두 번째 교훈은 단계적 구현의 필요성이다. 모든 기능을 한 번에 구현하려고 하면 오류가 발생했을 때 원인을 파악하기 어렵다. 본 프로젝트에서는 과제 추가, 목록 조회, 완료 처리, 삭제, 필터링 기능을 순서대로 구현하고 확인하였다. 이러한 방식은 오류를 줄이고, 기능별 테스트를 수행하기 쉽게 만들었다.

세 번째 교훈은 테스트와 캡처의 중요성이다. 기능이 구현되었다고 해서 곧바로 품질이 확보되는 것은 아니다. 실제로 버튼을 눌렀을 때 화면이 어떻게 바뀌는지, 사용자가 현재 상태를 이해할 수 있는지 확인해야 한다. 본 프로젝트에서는 기능별 테스트 결과 표와 실행 화면 캡처를 통해 각 기능의 정상 동작 여부를 확인하였다.

네 번째 교훈은 UI 품질도 개발 프로세스의 일부라는 점이다. 초기 필터 버튼은 기능적으로는 동작했지만, 선택된 버튼이 명확하게 보이지 않는 문제가 있었다. 이를 해결하기 위해 전체, 완료, 미완료 버튼을 파란색 계열로 변경하고, 선택된 버튼만 더 진하게 강조하였다. 이 과정에서 기능 구현뿐만 아니라 사용자 관점의 화면 개선도 품질 관리에 포함되어야 한다는 점을 확인하였다.

다섯 번째 교훈은 형상 관리와 기록의 중요성이다. 과제 조건에서는 착수 시점부터 제출 마감 전까지 지속적인 토의가 이루어졌음을 증빙해야 한다. 이를 위해 GitHub에 코드와 문서, 스크린샷을 정리하고, discussion-log와 prompt-log를 작성하였다. 이러한 기록은 개발 과정을 설명하는 자료가 되며, 막판에 한 번에 작업한 것이 아니라 단계적으로 개선했다는 근거가 된다.

6. 한계 및 개선 방향

본 프로젝트는 대학생 과제 관리에 필요한 기본 기능을 구현하였지만, 실제 서비스 수준의 완성도를 갖추기에는 몇 가지 한계가 있다. 첫째, 데이터 저장 방식이 JSON 파일 기반이므로 여러 사용자가 동시에 사용하는 환경에는 적합하지 않다. 향후에는 SQLite나 MySQL과 같은 데이터베이스를 적용하여 데이터 안정성과 확장성을 높일 수 있다.

둘째, 사용자 인증 기능이 없다. 현재는 단일 사용자를 기준으로 과제를 관리하는 구조이기 때문에 여러 사용자가 각자의 과제를 관리하는 웹앱으로 확장하려면 로그인 기능과 사용자별 데이터 분리 기능이 필요하다.

셋째, 알림 기능이 없다. 과제 마감일을 관리하는 앱이라면 마감일이 가까워졌을 때 알림을 제공하는 기능이 유용할 수 있다. 향후에는 마감일 기준 알림, 우선순위 표시, 과목별 분류 기능을 추가할 수 있다.

넷째, 테스트가 수동 테스트 중심으로 이루어졌다. 본 프로젝트에서는 실행 화면 캡처와 기능별 확인을 통해 테스트를 수행하였지만, 향후에는 자동화 테스트를 추가하여 기능 변경 후에도 기존 기능이 정상적으로 동작하는지 더 체계적으로 확인할 수 있다.

바이브코딩 측면에서도 개선할 점이 있다. 인공지능이 생성한 코드와 문서를 사용할 때는 그대로 복사하는 것이 아니라, 실제 실행 결과와 과제 조건에 맞게 검토하고 수정해야 한다. 또한 프롬프트를 체계적으로 작성하고, 각 단계에서 어떤 질문을 했는지 기록하면 개발 과정의 투명성을 높일 수 있다.

7. 결론

본 프로젝트는 ChatGPT 기반 바이트코딩을 활용하여 대학생 과제 관리 웹앱을 개발하고, 이를 소프트웨어 개발 프로세스 관점에서 분석한 사례이다. 프로젝트를 통해 요구사항 분석, 설계, 구현, 테스트, 품질 관리에 이르는 과정에서 바이트코딩이 개발 속도 향상, 오류 해결, 문서화 지원에 효과적이라는 점을 확인하였다.

그러나 바이트코딩은 개발자의 역할을 완전히 대체하지는 않는다. 요구사항을 명확히 정의하고, 생성된 코드를 실행하여 검증하며, UI 품질을 점검하고, 테스트 결과를 정리하는 과정은 여전히 개발자가 수행해야 한다. 특히 본 프로젝트에서는 필터 버튼의 선택 상태 표시, 입력값 검증, 포트 충돌 해결, 제출 자료 정리와 같은 부분에서 개발자의 판단과 반복적인 수정이 필요했다.

따라서 바이트코딩은 소프트웨어 개발 프로세스를 생략하게 만드는 방식이 아니라, 프로세스의 각 단계를 더 빠르고 효율적으로 수행하도록 돕는 보조 도구로 이해하는 것이 적절하다. 요구사항 분석에서 품질 관리까지의 절차를 유지하면서 바이트코딩을 활용할 때, 개발자는 더 짧은 시간 안에 실행 가능한 결과물을 만들 수 있고, 동시에 개발 과정의 품질과 추적 가능성을 확보할 수 있다.

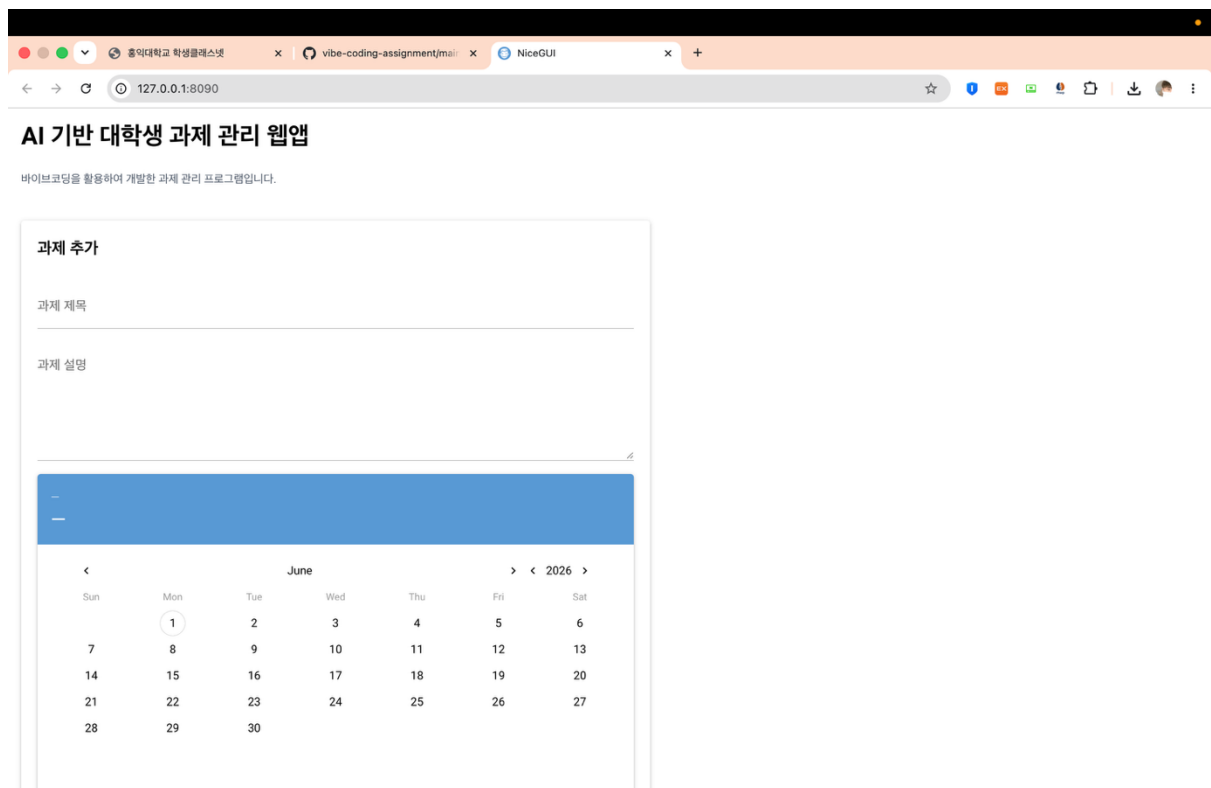
8. 부록

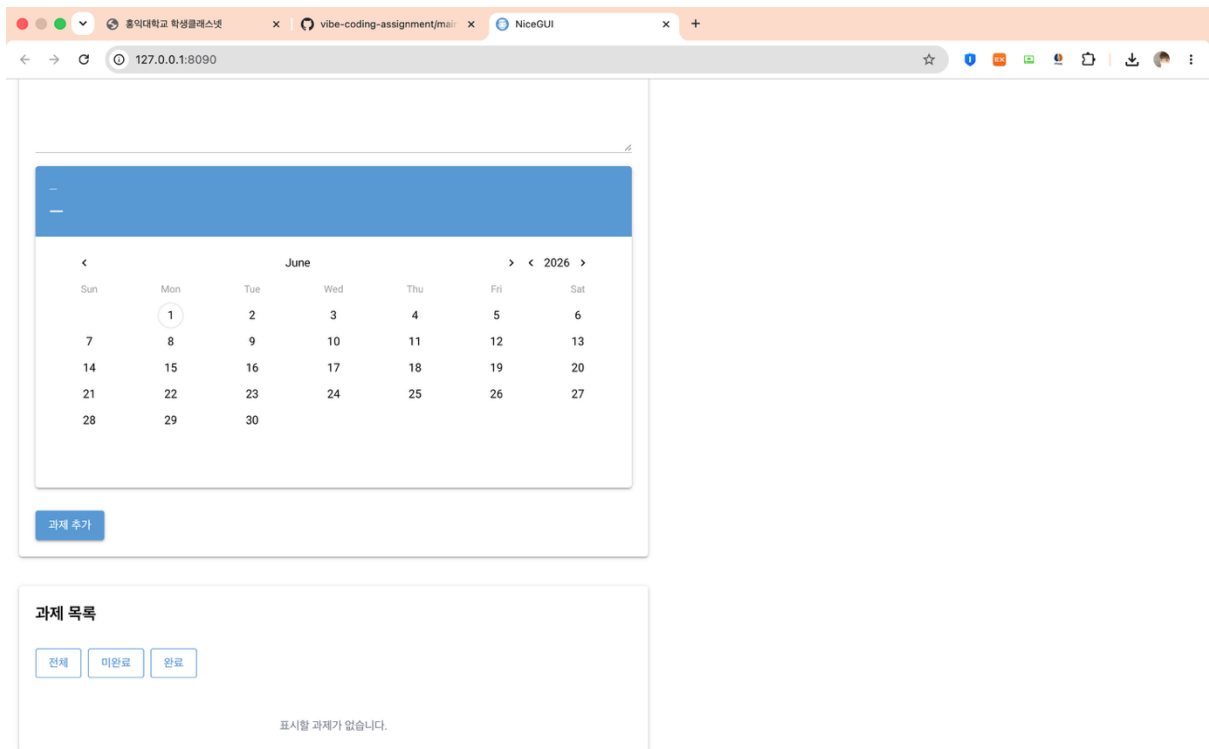
8.1 GitHub 저장소 주소

<https://github.com/LiebeEhre/vibe-coding-assignment>

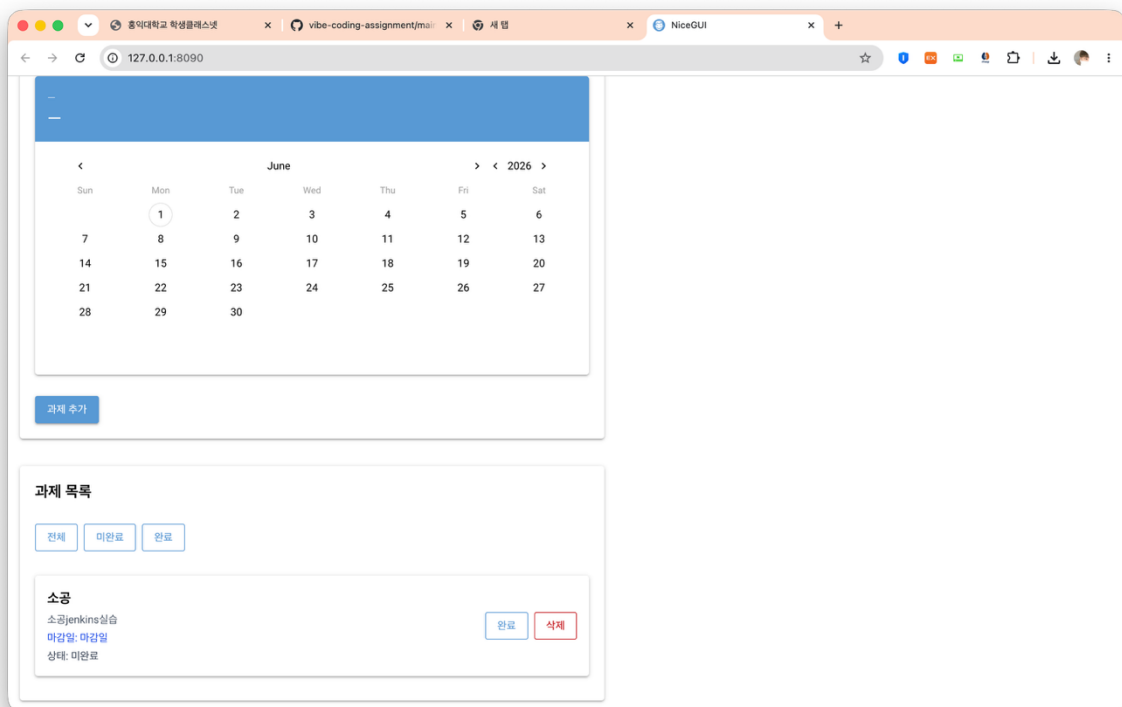
8.2 실행 화면 캡처 목록

1. Empty list

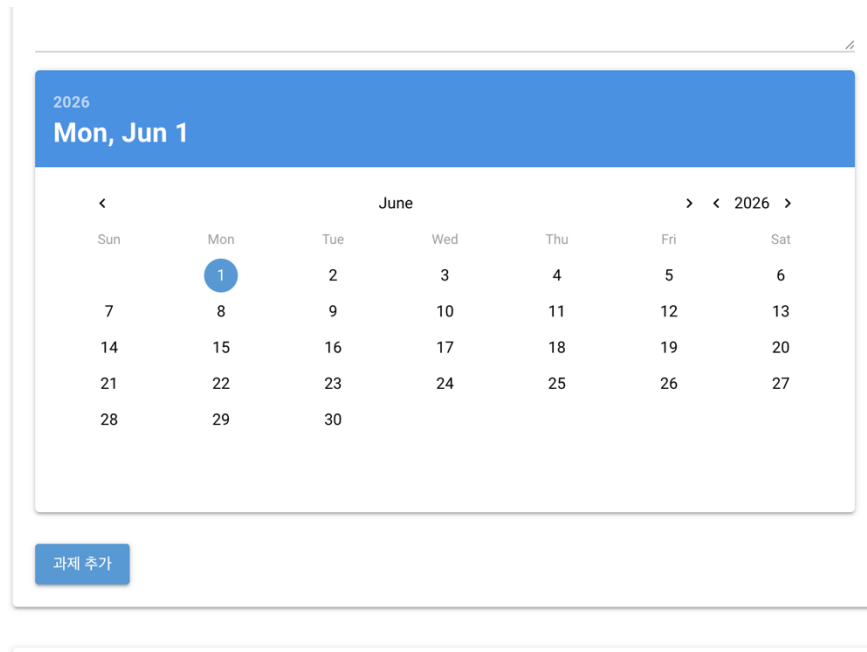




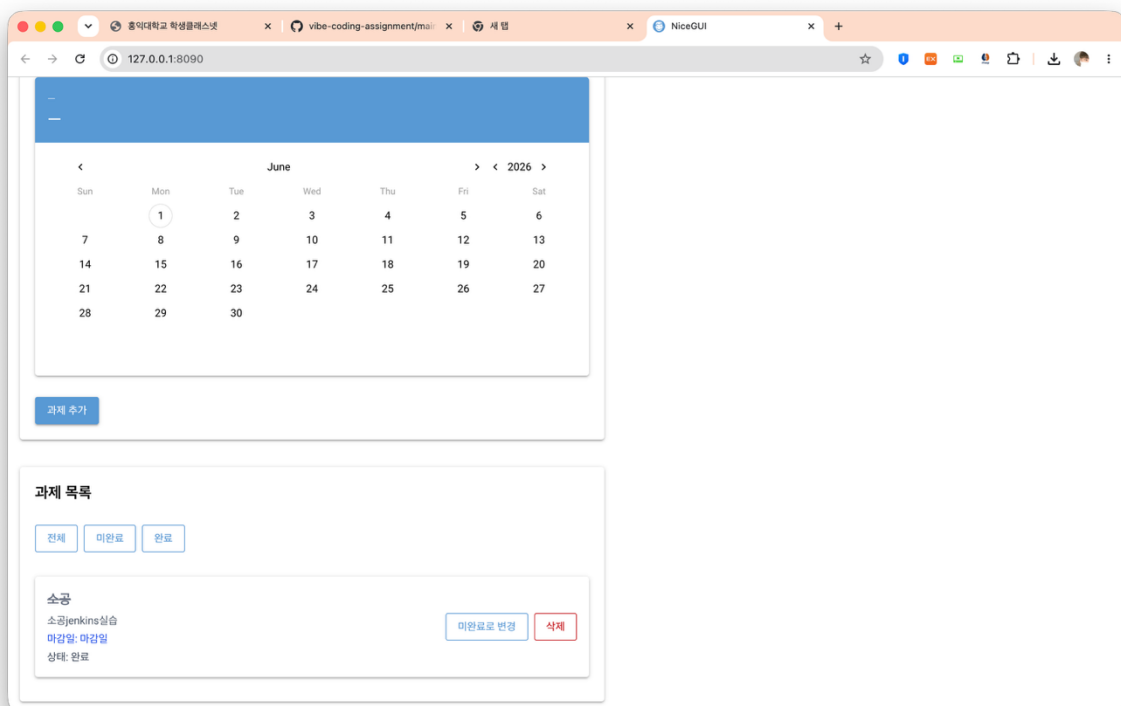
2. Add task



3. Calendar_date



4. Complete_task



5. Filter_all

전체

완료

미완료

소공

소공jenkins실습

마감일: 마감일

상태: 완료

미완료로 변경

삭제

CAhw3

hw3따라그리기

마감일: 2026-06-03

상태: 미완료

완료

삭제

시험

소공시험

마감일: 2026-06-08

상태: 미완료

완료

삭제

6. Filter_incomplete

과제 목록

전체

완료

미완료

CAhw3

hw3따라그리기

마감일: 2026-06-03

상태: 미완료

완료

삭제

시험

소공시험

마감일: 2026-06-08

상태: 미완료

완료

삭제

7. Filter_completed

과제 목록

전체

완료

미완료

소공

소공jenkins실습

마감일: 마감일

상태: 완료

미완료로 변경

삭제

8. Vaildation_empty_input

과제 추가

과제 제목

과제 설명

테스트

2026

Wed, Jun 3

<

June

> < 2026 >

Sun	Mon	Tue	Wed	Thu	Fri	Sat
	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30				

과제 추가

과제 제목을 입력하세요.

9. Delete_after

과제 추가

과제 목록

전체

완료

미완료

소공

소공jenkins실습

마감일: 마감일

상태: 완료

미완료로 변경

삭제

CAhw3

hw3따라그리기

마감일: 2026-06-03

상태: 미완료

완료

삭제

시험

소공시험

마감일: 2026-06-08

상태: 미완료

완료

삭제

과제가 삭제되었습니다.

8.3 프로세스 증빙 자료

프로세스 적용 증빙 자료는 GitHub 저장소에 함께 정리하였다.

- README.md: 프로젝트 개요, 실행 방법, 기능별 테스트 결과 정리
- discussion-log.md: 날짜별 논의 및 결정 사항 기록
- prompt-log.md: 바이트코딩 과정에서 사용한 프롬프트와 AI 응답 요약 기록
- screenshots/: 기능별 실행 화면 캡처 자료
- main.py: 과제 관리 웹앱 구현 코드
- tasks.json: 과제 데이터 저장